

Cost-Aware LLM Routing for AI Coding Assistants: A Two-Stage Stochastic and Robust Optimization Approach

1 Introduction

Modern AI-assisted coding environments such as Cursor rely on various large language models with different strengths, costs, and reliability profiles. A user may ask for help debugging code, solving a mathematical problem, answering a technical knowledge question, or generating software. In each case, the platform must decide which model should handle the prompt. Using a frontier model for every prompt may maximize quality, but it can be unnecessarily expensive. Conversely, using only the cheapest model may reduce cost but produce poor answers. This creates an operations research problem: how should a company like Cursor select a small pool of models and route prompts to those models while balancing cost, quality, and robustness?

The project prompt frames LLM routing as selecting a model $m \in M$ for each prompt $p \in P$, where each prompt-model pair has a cost $C(p, m)$ and a quality score $Q(p, m)$. The prompt also asks for a constrained optimization model that improves on a simple weighted cost-quality baseline by incorporating stochastic elements, multi-stage decisions, and realistic operational constraints.

In this project, we use Cursor as the AI company setting. The platform gained immense traffic in 2023 and that launch became a breakthrough in AI-pair programming. The platform made it easy and cheap to work with LLMs for all purposes. We further model Cursor’s routing problem as a two-stage stochastic optimization model with robust quality guarantees. The first-stage decision is to choose which models Cursor should maintain in its production pool. The second-stage decision is to route each prompt to one of the selected models. This structure matches the two-stage stochastic programming framework from class: a long-term decision is made first, and recourse decisions are made after uncertainty is realized. The final review notes for the course emphasize single-stage stochastic programming, two-stage stochastic programming, and robust optimization, so this formulation directly applies the course’s main modeling tools to a modern AI infrastructure problem.

The dataset contains 7,860 prompt-model evaluations, with 240 prompts, 33 candidate models, and four benchmark categories: AIME, GPQA, LCB, and MMLU-Pro. Each benchmark contributes 60 prompts. The score variable is binary and indicates whether a

model solved a prompt, while cost measures the cost of evaluating the prompt with that model.

This report answers two main business questions:

- How large should Cursor’s maintained model pool be before additional models have limited operational value?
- How much cost is required to guarantee higher average quality?

We also study robustness by analyzing how routing cost changes when model quality may be worse than measured on the benchmark.

2 Relationship to Course Modeling Techniques

This project builds directly on modeling techniques used throughout the course.

First, the routing decision is an assignment problem. In previous homework problems, binary variables were used to assign swimmers to strokes, arcs to paths, and counties to districts. Similarly, I define a binary variable x_{pm} that equals 1 if prompt p is assigned to model m . This follows the same assignment modeling logic used in the course.

Second, the model pool constraint is a resource constraint. In the nurse scheduling and chip production examples, decision variables were restricted by available capacity or production limits. Here, Cursor cannot maintain all 33 models because each model creates engineering overhead, including monitoring, integration, evaluation, billing, and provider management. Therefore, I impose a constraint limiting the number of selected models.

Third, the robustness experiment follows the same logic as the robust optimization homework. In that setting, uncertain coefficients were protected against unfavorable realizations. In this project, the uncertain parameter is model quality. If benchmark scores overestimate production performance, the router should still satisfy a minimum quality requirement.

At first, this looked like a simple cost-quality assignment model. After thinking more carefully about the uncertainty in future prompts, we interpreted the selected model pool as the longer-term decision and prompt routing as the adaptive decision. This helped us connect the project to assignment models, two-stage stochastic programming, and robust optimization rather than treating it as only a static routing problem.

Finally, the model-pool experiment is similar to the best-subset regression homework, where different subset sizes were compared. In this project, I vary the number of selected models K and study how cost, quality, and selected models change.

3 Data Description

Each row in the dataset corresponds to a prompt-model pair. The relevant columns are:

- **dataset:** benchmark category, one of AIME, GPQA, LCB, or MMLU-Pro
- **prompt_id:** unique identifier for a prompt
- **model:** model used to answer the prompt
- **score:** binary success indicator
- **cost:** cost of evaluating the prompt using that model

The data summary from the notebook confirms that the dataset has 7,860 rows, 240 unique prompts, and 33 unique models. Each of the four datasets contains exactly 60 prompts.

```

Dataset summary
-----
Rows: 7,860
Unique prompts: 240
Unique models: 33
Prompts by dataset:
dataset
AIME          60
GPQA          60
LCB           60
MMLU-Pro     60
Name: count, dtype: int64

```

Figure 1: Dataset summary and baseline results.

This data structure is ideal for an optimization model because every prompt-model pair has both a cost and a performance score.

4 Optimization Model

4.1 Sets

$P := \{1, \dots, N\},$	set of prompts, $N = 240,$
$M := \{1, \dots, J\},$	set of candidate models, $J = 33,$
$B := \{\text{AIME, GPQA, LCB, MMLU-Pro}\},$	set of benchmarks,
$P_b \subseteq P,$	set of prompts in benchmark $b \in B.$

4.2 Parameters

$C_{pm} \in \mathbb{R}_+$,	cost of assigning prompt $p \in P$ to model $m \in M$,
$Q_{pm} \in \{0, 1\}$,	quality score of model m on prompt p ,
$w_p \in \mathbb{R}_+$,	weight of prompt p ,
$K \in \mathbb{Z}_+$,	maximum number of selected models,
$Q_{\min} \in [0, 1]$,	minimum average quality requirement,
$Q_b^{\min} \in [0, 1]$,	minimum quality for benchmark $b \in B$,
$\delta \geq 0$,	robustness parameter.

Let w_p be the weight of prompt p in the sample average approximation. In the balanced empirical distribution,

$$w_p = \frac{1}{|P|}, \quad \forall p \in P.$$

4.3 Decision Variables

First-stage decision variables

$$z_m = \begin{cases} 1 & \text{if model } m \text{ is selected into the production pool,} \\ 0 & \text{otherwise,} \end{cases} \quad \forall m \in M.$$

Second-stage decision variables

$$x_{pm} = \begin{cases} 1 & \text{if prompt } p \text{ is routed to model } m, \\ 0 & \text{otherwise,} \end{cases} \quad \forall p \in P, m \in M.$$

The variables z_m represent long-term production planning decisions, while the variables x_{pm} represent routing decisions made after prompts are observed.

4.4 Objective Function

The main objective is to minimize the weighted average cost:

$$\min \sum_{p \in P} \sum_{m \in M} w_p C_{pm} x_{pm}.$$

In implementation, we add a small score-based tie-breaker:

$$\min \sum_{p \in P} \sum_{m \in M} w_p C_{pm} x_{pm} - \epsilon \sum_{p \in P} \sum_{m \in M} w_p Q_{pm} x_{pm},$$

where

$$\epsilon = 10^{-6}.$$

This is a lexicographic-style objective: cost minimization remains dominant, but among cost-equivalent solutions, the model prefers higher-quality routing policies. This is particularly important because many open-source models have zero cost, so multiple zero-cost routing policies may satisfy the same quality threshold.

4.5 Constraints

Assignment constraints. Each prompt must be assigned to exactly one model:

$$\sum_{m \in M} x_{pm} = 1 \quad \forall p \in P.$$

Activation constraints. A prompt can only be routed to a selected model:

$$x_{pm} \leq z_m \quad \forall p \in P, m \in M.$$

Model pool size constraint. At most K models may be selected:

$$\sum_{m \in M} z_m \leq K.$$

Average quality constraint. The average quality must exceed the production threshold:

$$\sum_{p \in P} \sum_{m \in M} w_p Q_{pm} x_{pm} \geq Q_{\min}.$$

Benchmark-level constraints. For each benchmark $b \in B$, the quality must exceed a minimum threshold:

$$\frac{1}{\sum_{p \in P_b} w_p} \sum_{p \in P_b} \sum_{m \in M} w_p Q_{pm} x_{pm} \geq Q_b^{\min} \quad \forall b \in B.$$

Binary constraints.

$$x_{pm} \in \{0, 1\}, \quad z_m \in \{0, 1\}.$$

5 Robust Formulation

Benchmark scores may not perfectly represent future production performance. To protect against overly optimistic estimates, define the *robust (pessimistic) score* as

$$\tilde{Q}_{pm} = \max(Q_{pm} - \delta, 0).$$

The robust quality constraint becomes

$$\sum_{p \in P} \sum_{m \in M} w_p \tilde{Q}_{pm} x_{pm} \geq Q_{\min}.$$

This follows the robust optimization framework from class: instead of relying on nominal parameter values, the model guards against adverse realizations. Larger values of δ represent greater uncertainty in model quality.

6 Stochastic and SAA Interpretation

The project prompt emphasizes that prompts should be treated as random variables drawn from an underlying distribution. The observed set of 240 prompts is therefore used to construct a *sample average approximation* (SAA) of the true distribution.

Under the balanced empirical distribution,

$$w_p = \frac{1}{240}, \quad \forall p \in P.$$

To evaluate sensitivity to future demand, we also solve reweighted versions of the problem corresponding to different prompt distributions:

- Balanced distribution,
- Coding-heavy distribution,
- Math-heavy distribution,
- Knowledge-heavy distribution.

This directly captures the idea that different products induce different demand patterns. For example, a coding assistant may receive a higher proportion of LCB-style prompts, while an educational tutoring system may place greater weight on AIME-style prompts.

Policy	Average Cost	Average Score	Models Used
Single Best Global Model	0.042548	0.8875	1
Cheapest per Prompt	0.000000	0.5667	1
Best Score per Prompt	0.001681	0.9833	24

Table 1: Baseline policy comparison.

7 Baseline Results

Before solving the proposed optimization model, we evaluate three baseline policies.

The single best global model (GPT-5) achieves a strong average score of 0.8875, but at a relatively high cost of 0.042548. The cheapest-per-prompt policy achieves zero cost but performs poorly, with an average score of only 0.5667. The best-score-per-prompt policy achieves the highest score (0.9833), but requires 24 different models, which is impractical for a production system due to significant engineering and maintenance overhead.

These baselines highlight the core trade-off. The objective is neither to maximize quality at any cost nor to minimize cost while ignoring performance, but rather to identify a practical operating point that balances cost, quality, and system complexity.

8 Recommended Production Router

The main recommended solution uses the parameters

$$K = 5, \quad Q_{\min} = 0.80, \quad \delta = 0.$$

The optimization model selects the following five models:

- GLM-Z1-9B-0414
- MiniCPM4.1-8B
- OpenThinker3-7B
- Qwen2.5-Coder-7B-Instruct
- internlm3-8b-instruct

The resulting performance is:

$$\text{Average cost} = 0.000000, \quad \text{Average score} = 0.8125.$$

Benchmark	Average Score
AIME	0.7333
GPQA	0.8833
LCB	0.8833
MMLU-Pro	0.7500

Table 2: Benchmark-level average scores for the recommended router.

Benchmark-level performance

The model performs particularly well on GPQA and LCB, both achieving an average score of 0.8833. This is especially relevant for a Cursor-style AI IDE, where strong performance on coding-related tasks is critical. AIME and MMLU-Pro performance is comparatively lower but still satisfies the benchmark-level quality constraints.

9 Routing Distribution

Table 3 shows the routing distribution for

$$K = 5, \quad Q_{\min} = 0.80, \quad \delta = 0.$$

Model	Routed Prompts
GLM-Z1-9B-0414	71
MiniCPM4.1-8B	59
OpenThinker3-7B	46
Qwen2.5-Coder-7B-Instruct	34
internlm3-8b-instruct	30

Table 3: Routing distribution across selected models.

This result shows that the selected model pool is not used uniformly. Instead, different models specialize in different subsets of prompts. The optimizer routes the largest share of prompts to GLM-Z1-9B-0414, while the remaining models are assigned prompts where they provide higher quality at comparable cost.

The result also showcases that a compact but diverse model pool can outperform a single model by exploiting specialization across different prompt types.

10 Experiment 1: Model Pool Size

The first business question is:

How many models should Cursor maintain?

We solve the model for

$$K = 1, 2, \dots, 8, \quad \text{with} \quad Q_{\min} = 0.80.$$

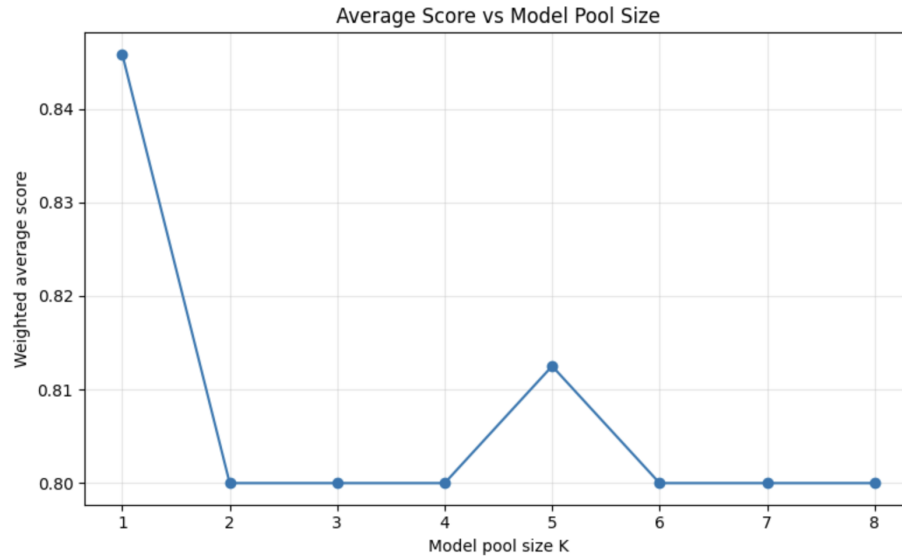


Figure 2: Average score vs model pool size.

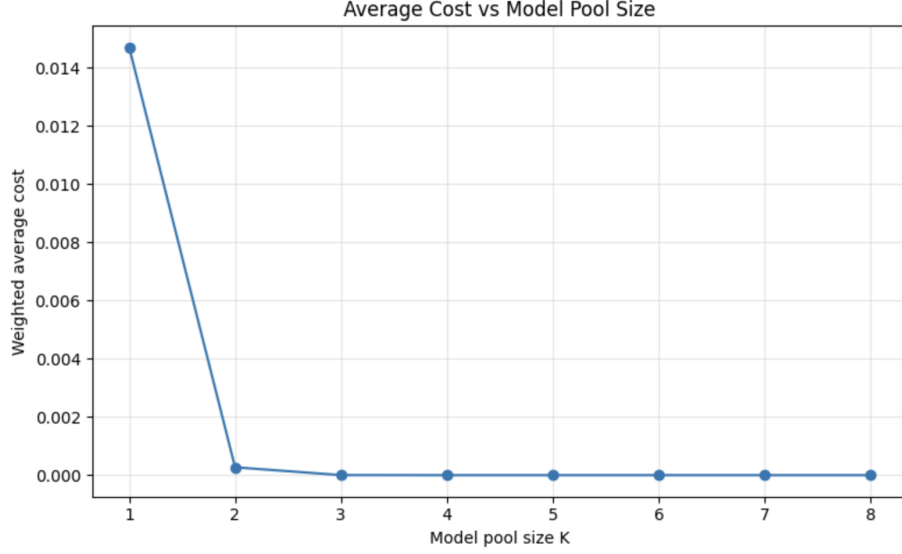


Figure 3: Average cost vs model pool size.

The cost plot provides the clearest interpretation. With $K = 1$, the optimizer must use a stronger paid model to satisfy the quality threshold, resulting in average cost approximately 0.014699. As K increases, the optimizer has more flexibility to combine specialized models. By $K = 4$, the optimizer can satisfy $Q_{\min} = 0.80$ at zero average cost.

The score plot is not monotone, but this is not a modeling error. The objective minimizes cost subject to a quality threshold; it does not maximize score. Therefore, once the model satisfies

$$\text{average score} \geq 0.80,$$

there is no incentive to increase score unless it can be done at the same cost.

The main conclusion is that a small model pool is sufficient. Cursor does not need to maintain all 33 models. A pool of around 4–5 models can satisfy the target quality threshold at essentially zero cost.

11 Experiment 2: Cost-Quality Frontier

The second business question is:

How much cost is required to guarantee higher quality?

We fix

$$K = 5$$

and vary

$$Q_{\min} \in \{0.70, 0.75, 0.80, 0.85, 0.90\}.$$

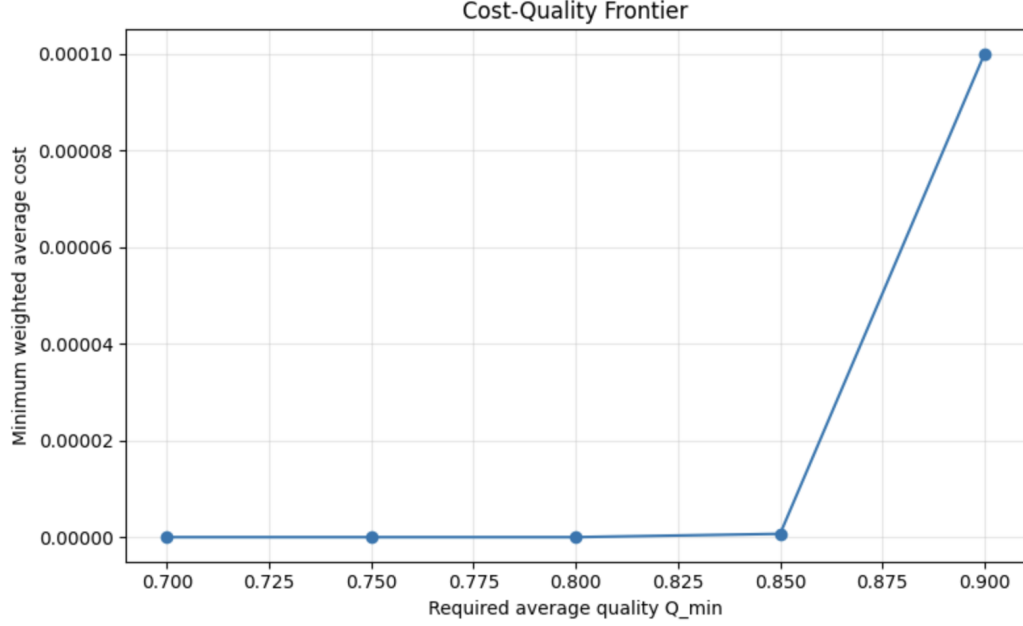


Figure 4: Cost-quality frontier.

For $Q_{\min} = 0.70, 0.75$, and 0.80 , the optimizer satisfies the quality target at zero cost. At $Q_{\min} = 0.85$, cost becomes positive but remains small. At $Q_{\min} = 0.90$, cost increases sharply relative to the lower thresholds.

A quality target around 0.80 is a cost-efficient operating point. If Cursor requires 0.90 average quality, the system must rely more heavily on stronger or paid models, leading to a measurable increase in cost.

12 Experiment 3: Cost of Robustness

To study robustness, we fix

$$K = 5, \quad Q_{\min} = 0.80$$

and vary

$$\delta \in \{0, 0.02, 0.05, 0.08, 0.10\}.$$

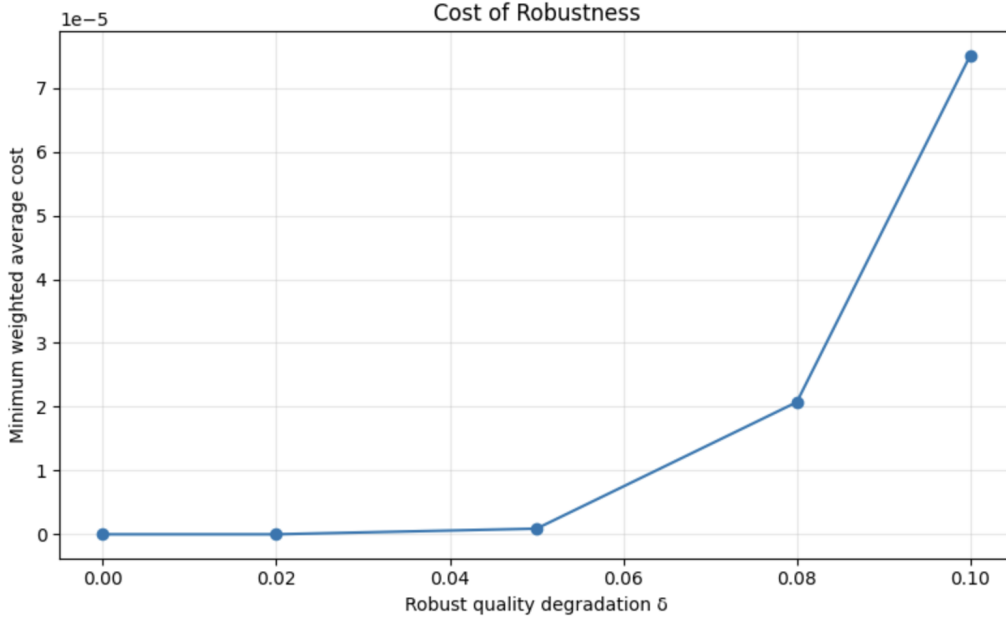


Figure 5: Cost of robustness.

δ	Average Cost	Nominal Score	Robust Score
0.00	0.000000	0.8000	0.8000
0.02	0.000000	0.8167	0.8003
0.05	0.000000886	0.8458	0.8035
0.08	0.000020725	0.8708	0.8012
0.10	0.000075096	0.8917	0.8025

The interpretation is consistent with robust optimization theory. When the degradation parameter δ is small, low-cost models are sufficient. As δ increases, the optimizer is forced to select stronger models so that the degraded performance still satisfies the constraint $Q_{\min} = 0.80$, which increases average cost.

Thus, robustness improves reliability, but it is not free.

13 Experiment 4: Prompt Distribution Sensitivity

The final experiment studies how the solution changes when the prompt distribution changes. This is important because the empirical dataset may not match future Cursor usage. We test four prompt distributions:

Scenario	Average Cost	Average Score	Models Selected
Balanced	0.0000	0.8000	5
Coding-heavy	0.0000	0.8008	5
Math-heavy	0.0000	0.8033	4
Knowledge-heavy	0.0000	0.8025	5

The results show that the model remains cost-efficient across different prompt distributions. However, benchmark-level performance shifts depending on which dataset receives higher weight. For example, the coding-heavy case prioritizes LCB performance, while the math-heavy case improves AIME performance.

This demonstrates that the formulation can adapt to different product settings simply by changing the prompt weights w_p .

14 Discussion and Recommendations

The results support several conclusions listed below:

- **Cheapest model is insufficient:** The cheapest-per-prompt policy has zero cost but only achieves an average score of 0.5667, which is too low for production use.
- **Best global model is too expensive:** GPT-5 achieves an average score of 0.8875, but at a high cost of 0.042548 per prompt.
- **Unconstrained best-score routing is impractical:** While it achieves an average score of 0.9833, it uses 24 different models, creating significant operational overhead (integration, monitoring, fallback logic, and provider management).
- **Proposed two-stage model is a strong compromise:**
 - Uses only 5 selected models
 - Achieves average score 0.8125
 - Maintains essentially zero average cost
 - Outperforms the cheapest baseline while avoiding frontier-model costs
- **Robust optimization improves reliability:** As the degradation parameter δ increases, the optimizer shifts toward stronger (more expensive) models to maintain performance under uncertainty.
- This provides a direct mechanism to trade off cost vs. reliability in production.

Final recommendation:

- Maintain a compact pool of approximately 5 lightweight models for standard routing.
- Use this pool for most everyday queries to achieve low cost with acceptable quality.
- Reserve frontier or paid models for:
 - High-quality threshold requirements
 - High-stakes prompts
 - Robust fallback under uncertainty ($\delta > 0$)

15 Limitations and Future Work

There are several limitations of this work:

- **Binary score variable:** The score Q_{pm} is binary and does not capture partial correctness, response quality, helpfulness, latency, or user satisfaction.
- **Limited benchmark representation:** The dataset is only a sample of future production demand. Real Cursor usage may include longer, codebase-specific, or multi-turn tasks that are not fully represented by AIME, GPQA, LCB, or MMLU-Pro.
- **No latency modeling:** The current formulation does not include inference latency. In practice, a model may be accurate and low-cost but too slow for interactive coding applications.
- **Static routing policy:** The routing decision is fixed after optimization. A production system would ideally adapt dynamically to changes in user feedback, model availability, cost fluctuations, and performance drift.

Future extensions could include:

- Latency-aware constraints and multi-objective optimization (cost, quality, latency)
- Context-window limitations for long-form coding tasks
- Provider diversity constraints to reduce single-vendor dependence
- Memory/storage constraints for local or cached models
- Cascading or fallback routing architectures
- Online learning or reinforcement learning-based routing policies

16 Personal Takeaway

This project showed that LLM routing is not only a machine learning problem but also an operations research problem. The main challenge is not simply selecting the “best” model, but rather balancing cost, quality, robustness, and operational complexity.

The most interesting result is that a small pool of lightweight models can achieve a useful quality threshold at zero cost. The cheapest model alone is not sufficiently strong, and the best-score oracle uses too many models to be operationally realistic. The optimization model therefore identifies a practical middle ground that better reflects real production constraints.

Overall, this project connects several core course concepts—assignment variables, two-stage stochastic programming, sample average approximation, and robust optimization—to a modern AI systems application in LLM routing.

17 Code Availability

The Python/Pyomo code used to solve the optimization model and generate the tables and figures in this report is available at:

https://github.com/alakhanpal23/IND_ENG164-Project